



Programação Avançada 2025/26

Introdução ao Refactoring

Bruno Silva, Patrícia Macedo

Sumário

- Refactoring
 - Conceitos:
 - Refactoring;
 - *Code Smells* (ou *Bad Smells*);
 - Exemplo;
 - Catálogo de *Code Smells* e Técnicas de Refactoring.
 - Atividade.

Conceitos

Refactoring é a alteração de um programa de software, através da melhoria da sua estrutura interna (desenho) e/ou código, sem alterar o seu comportamento (i.e., assegurando a manutenção da sua funcionalidade).

- O processo de **refactoring**:
 - Trata um "problema" de cada vez;
 - Preserva a correção do programa;
 - Deve recorrer ao uso de testes unitários.
 - Não deve criar funcionalidade adicional.

Conceitos

Code Smell (ou *Bad Smell*) são certas "estruturas" no código que indicam violação de princípios fundamentais de desenho e que influenciam negativamente a qualidade do código.

- Os **code smells**:
 - Não são *bugs*, porque não impedem o funcionamento correto do programa;
 - São potenciais "fraquezas" no desenho/código que podem influenciar negativamente a manutenção corretiva e evolutiva e aumentar, sim, o risco de *bugs* no futuro.
 - ⚠ São alertas para averiguar necessidade de *refactoring*.

Conceitos

A temática do *refactoring* foi estudada aprofundadamente [1], pelo que existe um catálogo de *code smells* típicos/frequentes.

Para cada um destes *bad smells* existe uma **técnica de refactoring** apropriada.

Uma **técnica de refactoring** é, portanto, uma solução proposta e estabelecida para resolver um *code smell* (fraqueza) por forma a melhorar a qualidade do código/desenho segundo bons princípios.

Processo de Refactoring

! Idealmente devem existir testes unitários para garantir o correto funcionamento do programa.

- Estes também servem para garantir a correção após quaisquer alterações do código.

1. 🔍 Procurar no código um potencial **code smell**;
2. 🛠️ Aplicar a **técnica de refactoring** apropriada;
3. ! Garantir a correção do programa, executando os **testes unitários**.
4. ↶ Voltar ao passo 1 até que todos os *code smells* tenham sido tratados.

Vantagens ✨

- Melhora o desenho do programa de software;
- Torna o código mais eficiente;
- Torna o código mais fácil compreender e de manter;
 - Na maioria dos casos, outras pessoas que não desenvolveram o código terão que o manter.
- Reduz a propensão a erros futuros (bugs).

Exemplo (1/4)

Considere o seguinte código:

```
public class Animal {  
    enum TYPE_ANIMAL {MAMMAL, BIRD, REPTILE};  
    TYPE_ANIMAL myKind; // set in constructor  
    ...  
    String getSkin() {  
        switch (myKind) {  
            case MAMMAL: return "hair";  
            case BIRD: return "feathers";  
            case REPTILE: return "scales";  
            default: return "integument";  
        }  
    }  
}
```

? Problema que este código apresenta em termos de desenho e futura manutenção corretiva e evolutiva?

Exemplo (2/4)

Code Smell: *Switch Statement*

Técnica de Refactoring: *Replace Conditional with Polymorphism*

```
public abstract class Animal {  
    ...  
    public abstract String getSkin();  
}  
  
public class Mammal extends Animal {  
    String getSkin() { return "hair"; }  
}  
  
public class Bird extends Animal {  
    String getSkin() { return "feathers"; }  
}  
  
public class Reptile extends Animal {  
    String getSkin() { return "scales"; }  
}
```

Exemplo (3/4)

Porque é que este *refactoring* foi uma melhoria?

- Adicionar um novo tipo de animal, não implica alterar *rever* e recompilar o código existente;
- Como *Mammals*, *Birds*, e *Reptiles* são animais que podem ser diferenciados de outras formas, assim evitaremos outros switch's.
- Utilizamos bons princípios de orientação a objetos (herança e polimorfismo).

Exemplo (4/4)

Com a existência de **testes unitários**, os mesmos devem completar com sucesso antes e após o *refactoring*, e.g.,:

```
public void testGetSkin() {  
    assertEquals("hair", myMammal.getSkin());  
    assertEquals("feathers", myBird.getSkin());  
    assertEquals("scales", myReptile.getSkin());  
    assertEquals("integument", myAnimal.getSkin());  
}
```

Catálogo de *Code Smells* e Técnicas de Refactoring



Code Smells

<https://refactoring.guru/refactoring/smells>

Categorias: *Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables, Couplers, Other Smells.*



Técnicas de Refactoring

<https://refactoring.guru/refactoring/techniques>

Categorias: *Composing Methods, Moving Features between Objects, Organizing Data, Simplifying Conditional Expressions, Simplifying Method Calls, Dealing with Generalization.*

Catálogo de *Code Smells* e Técnicas de Refactoring

Code Smells

Cada *code smell* apresenta o seguinte:

- **Nome:** Nome do code smell;
- **Sinais e Sintomas:** Descrição sucinta do potencial problema;
- **Razões:** Justificação do problema;
- **Tratamento:** De acordo com a situação concreta onde o code smell existe, sugere a técnica de refactoring apropriada.

Catálogo de *Code Smells* e Técnicas de Refactoring

Técnicas de Refactoring

Cada técnica de refactoring apresenta o seguinte:

- **Nome:** Nome da técnica;
- **Problema:** Descrição sucinta da situação que faz com que seja preciso aplicar a técnica;
- **Motivação:** O que se vai ganhar com a aplicação da técnica;
- **Solução:** Descrição de aplicação da técnica;
- **Code Smells:** Que *code smells* podem ser tradados pela técnica.

Exemplo | Técnica Refactoring (*)

- **Name:** Replace Conditional with Polymorphism
- **Summary:** You have a conditional that chooses different behavior depending on the type of an object.
- **Mechanism:** Move each leg of the conditional to an overriding method in a subclass. Make the original method abstract.
- **Example:** (apresentado anteriormente)

```
public class Animal {  
    enum TYPE_ANIMAL {MAMMAL,BIRD,REPTILE} ;  
    TYPE_ANIMAL myKind; // set in constructor  
    ...  
    String getSkin() {  
        switch (myKind) {  
            case MAMMAL: return "hair";  
            case BIRD: return "feathers";  
            case REPTILE: return "scales";  
            default: return "integument";  
        }  
    }  
}
```

```
public abstract class Animal {  
    ....  
    public abstract String getSkin();  
}  
  
public class Mammal extends Animal {  
    String getSkin() { return "hair"; }  
}  
  
public class Bird extends Animal {  
    String getSkin() { return "feathers"; }  
}  
  
public class Reptile extends Animal {  
    String getSkin() { return "scales"; }  
}
```

Atividade (🕒 15 min + 20 min)

1. A turma é dividida em grupos e a cada grupo é atribuído uma letra:

- A : message chain + temporary field;
- B : data clump + duplicate code;
- C : primitive obsession + data class;
- D : refused bequest + inappropriate intimacy

2. Para cada uns dos **CODE SMELLS**:

- Procurar o seu significado;
- Escrever código que seja um exemplo do CODE SMELL.

3. Apresentar o resultado à turma.

💡 **Referência:** <https://refactoring.guru/refactoring/smells> | mas pode fazer pesquisas adicionais.

Revisão

Algumas definições

- **Refactoring** (*substantivo*): mudança feita na estrutura interna do programa de software para facilitar a sua compreensão e melhorar a sua estrutura interna.
- **Refactor** (*verbo*): acto de reestruturar o programa de software aplicando uma série de técnicas de refactoring sem alterar seu comportamento observável.
- **Code smells**: Sinais de aviso no código de má programação e/ou desenho.

Discussão 🤔

- Determinar o que é e o que não é um *code smell* é subjetivo e varia de acordo com a linguagem, o programador e o paradigma de programação.
- Toda a temática carece de espírito crítico, mas, não obstante, devemos assentar o processo de refactoring no corpo de trabalho já desenvolvido por peritos;
- Poderão existir *code smells* que não terão tratamento, i.e., como consequência de outras opções de desenho, e.g., **data classes** dentro do padrão de desenho DAO (ou framework ORM).

Bibliografia

[1] Fowler, Martin. Refactoring: Improving the Design of Existing Code. Addison-Wesley, 2nd Edition, 2018.

[2] <https://refactoring.guru/refactoring>

[3] <https://refactoring.com/catalog/>